

Julia port of the FSA-v2 model: closed-loop SMC² filter + GPU controller

Coding session writeup

2026-05-07

1 The Julia FSA-v2 codebase

1.1 Repository layout

The model lives at `version_2_Julia/models/fsa_high_res/` and is organised as eight files that mirror the Python source under `version_2/models/fsa_high_res/`. The aggregator module `FSAHighRes.jl` includes them in the order required by the inter-file using-clauses (`PhiBurst` before `Simulation` because the latter reads its constants; `CPU control` before `GPU control` because the latter re-uses some of the same RBF helpers; `estimation` last because it depends on the framework's `EstimationModel`).

File	Purpose
<code>_dynamics.jl</code>	SDE drift + state-dependent diffusion (Stage G1 reparametrisation). Pure Julia; takes a NamedTuple params for ergonomic <code>params.tau.B</code> access.
<code>_phi_burst.jl</code>	Daily- $\Phi \rightarrow$ sub-daily morning-loaded burst envelope. Reads <code>FSA_STEP_MINUTES</code> env var to set <code>BINS_PER_DAY</code> .
<code>simulation.jl</code>	<code>INIT_STATE</code> , <code>DEFAULT_PARAMS</code> , the four observation samplers (HR / sleep / stress / steps), circadian $C(t)$.
<code>_plant.jl</code>	<code>StepwisePlant</code> struct. Holds truth params, current (B, F, A) state, RNG seed offset and a <code>history</code> dict. Two advance methods: <code>advance!</code> (takes daily Φ , applies burst envelope) and <code>advance_subdaily!</code> (takes per-bin Φ directly). <code>finalise</code> writes a psim-format artefact.
<code>cpu_control.jl</code>	CPU-side cost-functional builder. Produces a <code>ControlBundle</code> of <code>(decoder, cost_fn, traj_sample_fn)</code> closures used by the framework's CPU <code>run_tempered_smc_loop</code> . Currently unused in the production path (the GPU controller superseded it) but kept for debugging. Hosts the canonical <code>build_rbf</code> .
<code>estimation.jl</code>	Priors (30 estimated + 5 frozen), the locally guided (Pitt–Shephard) <code>propagate_fn</code> that fuses three Gaussian channels via sequential scalar Kalman, the Bernoulli sleep weight, the <code>align_obs_fn</code> that puts everything on a <code>Float32</code> per-bin grid, and the <code>build_estimation_model</code> factory.
<code>gpu_pf.jl</code>	FSA-specific GPU bootstrap-PF kernel (<code>fsa_propagate_segment_kernel!</code>) + <code>FSAGPUTargetBatched</code> struct. Calls into the framework-shared <code>run_segmented_smc_step!</code> for resampling / Liu–West / OT rescue.
<code>gpu_control.jl</code>	FSA-specific GPU cost kernel (<code>fsa_cost_kernel!</code>) + <code>FSAControlGPUPUTarget</code> struct + <code>make_log_density_fn</code> closure factory. Hands a closure to the framework's <code>run_tempered_smc_gpu</code> .
<code>FSAHighRes.jl</code>	Module aggregator. includes the eight files and re-exports the public symbols.

1.2 Stage-G1 reparametrisation in `_dynamics.jl`

The state $\mathbf{X} = (B, F, A)$ evolves under the Itô SDE

$$dB = \kappa_B \cdot \frac{1 + \varepsilon_A A}{1 + \varepsilon_A A_{\text{typ}}} \cdot \Phi dt - \frac{B}{\tau_B} dt + \sigma_B \sqrt{B(1-B)} dW_B \quad (1)$$

$$dF = \kappa_F \Phi dt - \frac{1 + \lambda_A A}{1 + \lambda_A A_{\text{typ}}} \cdot \frac{F}{\tau_F} dt + \sigma_F \sqrt{F} dW_F \quad (2)$$

$$dA = \mu(B, F) A dt - \eta A^3 dt + \sigma_A \sqrt{A} dW_A \quad (3)$$

$$\mu(B, F) = \mu_0 + \mu_B B - \mu_F F - \mu_{FF} (F - F_{\text{typ}})^2, \quad (4)$$

with the operating-point constants $A_{\text{typ}} = 0.10$, $F_{\text{typ}} = 0.20$. The reparametrisation absorbs three rank-deficient direction pairs $(\kappa_B, \varepsilon_A)$, (μ_F, μ_{FF}) , (τ_F, λ_A) into strongly-identified directions and weakly-identified residuals. Truth values use the effective parameters: $\kappa_B^{\text{eff}} = 0.01248$, $\tau_F^{\text{eff}} = 7/1.1 = 6.364$, $\mu_F^{\text{eff}} = 0.26$, $\mu_0^{\text{eff}} = 0.036$. The Julia `TRUTH_PARAMS` NamedTuple is constructed from these identities so it is bit-equivalent to Python's `simulation.py:DEFAULT_PARAMS`.

The substepped Euler–Maruyama integrator `em_step_substepped` runs `n_substeps=4` drift sub-steps of length $\Delta t/4$, then applies one Wiener increment of variance $\sigma(y_{\text{det}})^2 \cdot \Delta t$ at the outer-bin boundary, then performs boundary reflection ($B \mapsto -B$ if $B < 0$, $B \mapsto 2 - B$ if $B > 1$, $F \mapsto |F|$, $A \mapsto |A|$). This matches Python’s `imex_step_substepped` exactly.

1.3 Burst envelope (`_phi_burst.jl`)

The controller plans daily Φ values; the plant runs the SDE on a sub-daily grid (`BINS_PER_DAY = (60 · 24) / FSA_ST`). The expansion is a Gamma-shaped morning-loaded envelope:

$$\Phi(t) = \Phi_d \cdot e(k), \quad e(k) \propto t_{\text{post}} e^{-t_{\text{post}}/\tau_h}$$

in the wake window [7h, 23h], normalised so $\sum_k e(k) \cdot \Delta t_h = 24$. The integral preservation makes the slow Banister dynamics (which respond only to the daily integral) indistinguishable between bursty and smooth Φ , but the fast dynamics (CIR for F , Stuart–Landau for A) see the bursts. This matters in benchmarks — see §??.

1.4 Stepwise plant (`_plant.jl`)

`StepwisePlant` is a mutable Julia struct carrying `state::Vector{Float64}`, `t_bin::Int` (global bin counter), the truth params and the per-channel observation history. Its two driver methods are

- `advance!(plant, stride_bins, Phi_daily)` — expects daily Φ , expands to sub-daily via `expand_daily_phi_to_subdaily`, then runs the per-bin SDE.
- `advance_subdaily!(plant, Phi_subdaily)` — takes per-bin Φ values directly, bypasses the burst envelope. Added so the GPU controller’s per-bin RBF schedule can be applied.

After advancing, `finalise` writes `manifest.json`, `trajectory.npz`, per-channel `obs/*.npz` and `exogenous/*.npz` so the closed-loop run can be replayed by the Python `psim` consistency checks.

1.5 Estimation model (`estimation.jl`)

The 30 estimated parameters are declared as a `Vector{Tuple{Symbol, PriorType}}` with the same order as Python’s `PARAM_PRIOR_CONFIG` so the posterior cloud aligns column-by-column. The locally-guided proposal (`propagate_fn`) computes a Gaussian prior predictive $\mathcal{N}(\mu_{\text{pred}}, P_{\text{pred}})$ from the G1 drift and state-dependent variance, then sequentially fuses the three Gaussian observation channels (HR, stress, $\log(\text{steps} + 1)$) by scalar Kalman updates gated by the `*_present` masks. A Cholesky-3 factorisation samples x_{new} , and the predictive log-marginal is returned as the importance correction `pred_lw`. The Bernoulli sleep log-likelihood is added by `obs_log_weight_fn`. This is a line-by-line port of Python’s `estimation.py:propagate_fn`.

1.6 GPU particle filter (`gpu_pf.jl`)

The model file contains only the FSA-specific propagate kernel — exactly the same locally-guided proposal as the CPU version, written as a `KernelAbstractions @kernel` that runs in fp32 with one thread per (chain, particle). The framework-shared kernels for per-chain weight statistics, normalised-weights cumulative sum, systematic resampling with Liu–West shrinkage, and Sinkhorn-based OT rescue live in `julia/SMC2FC/src/Filtering/GPUSegmentedPF.jl`. A driver routine `run_segmented_smc_step!` composes them. The model’s only entry point is `gpu_log_density_batched(target, U)` returning per-chain log-marginal-likelihood used by the outer SMC² HMC.

1.7 GPU controller (`gpu_control.jl`)

The control kernel `fsa_cost_kernel!` has one thread per (chain m , common-random-number trial t). Per outer bin k :

1. Decode $\Phi(t_k)$ from the chain's θ_m via the raw Gaussian RBF basis, $\Phi(t_k) = \Phi_{\max} \cdot \sigma(c_\Phi + \sum_a \theta_{m,a} \varphi_a(t_k))$ with $c_\Phi = \text{logit}(\Phi_{\text{def}}/\Phi_{\max})$.
2. PRE-step accumulate the cost integrand: $A_{\text{acc}} += A \Delta t$, $\Phi_{\text{acc}} += \Phi^2 \Delta t$, $B_{\text{acc}} += \max(F - F_{\max}, 0)^2 \Delta t$.
3. Run the substepped EM SDE step with the trial-shared CRN noise.

At the end of the bin loop the cost $J_m = -A_{\text{acc}} + \lambda_\Phi \Phi_{\text{acc}} + \lambda_F B_{\text{acc}}$ is written to `cost_per_thread[i]`.

The outer wrapper averages over trials. The model exposes a `make_log_density_fn(target)` closure that the framework's generic `run_tempered_smc_gpu` (in `julia/SMC2FC/src/Control/GPUControlSMC.jl`) consumes. The framework provides parallel-chains HMC, a ChEES leapfrog picker, FD-batched gradients and the tempered-SMC² outer loop; none of those are FSA-specific.

1.8 Bugs found while debugging the controller

Five issues turned up as the bench was iterated against the Python plot. The first four are fixed in the Julia source; the fifth is identified but not yet patched.

(1) RBF basis was row-normalised. Python's `smc2fc/control/rbf_schedules.py` returns the raw Gaussian basis; the original Julia port row-normalised it into a partition of unity. Row-normalisation collapses any per-anchor θ variation into a weighted average, so the decoded $\Phi(t)$ was effectively constant regardless of θ — the reason the controller produced flat Φ schedules. Removing the normalisation in both `cpu_control.jl:build_rbf` and the GPU target's RBF block moved the controller's optimum from $\Phi \approx 0.45$ flat to $\Phi \approx 0.20$ flat (matching the spec text in §10.2 of the FSA-v2 outdated lecture notes).

(2) NaN guard reset state. The original kernel had an `if !isfinite(B)...reset to (0.05, 0.30, 0.10)` guard at the end of every bin. When fp32 rounding produced occasional small non-finite intermediates, this silently zeroed out cost differences between schedules. Removed.

(3) $+\varepsilon_A$ inside cost diffusion. The plant's `noise_scale_fn` uses `sqrt(A + EPS_A_FROZEN)` to keep the diffusion well-defined at $A = 0$; the controller's `diffusion_state_dep` uses `sqrt(max(A, 0))`. The Julia port had carried the eps form into the cost kernel. Aligned to Python's no-eps form for the controller path; plant kept on the eps form.

(4) POST-step cost accumulation. The kernel was accumulating $A_{\text{acc}} += A \Delta t$ AFTER the EM step. Python uses the PRE-step state (`y[2]`, not `y_next[2]`), i.e. a left-Riemann sum from $t = 0$ to $t = T - \Delta t$. Switched.

(5) Burst envelope asymmetry between MPC plant and baseline plant. The benchmark calls `advance!(base_plant, ..., [1.0])` for the constant- Φ baseline (which expands $\Phi_{\text{daily}} = 1$ through the burst envelope into per-bin morning-loaded peaks of ~ 2 – 3) but `advance_subdaily!(mpc_phi_per_bin)` for the MPC plant (which feeds the controller's smooth per-bin Φ directly, skipping the burst envelope). A diagnostic Monte-Carlo with $n = 32$ seeds confirmed that under *constant* smooth $\Phi = 1$, the plant's mean $\bar{A} = 0.1002$, but the spec quotes $\bar{A} = 0.0823$ for bursty $\Phi_{\text{daily}} = 1$. The 22% gap is exactly the burst-envelope asymmetry. The fix is to aggregate the controller's per-bin schedule to daily Φ values and call `advance!` with the burst envelope, so MPC and baseline run on the same fast-dynamics regime.

1.9 Outstanding for next iteration

- Apply the burst-envelope fix (5) so MPC and baseline trajectories are produced under the same fast-scale forcing.
- Switch the controller’s gradient from central FD ($h = 10^{-4}$, 17 chain-evals per gradient) to ForwardDiff dual-number autodiff. Per the bistable B3 module’s empirical note, ForwardDiff with $d = 8$ out-performs Enzyme reverse-mode by $\sim 2.7\times$ on a similar PF; the FSA controller has the same dimensionality so the same conclusion should hold.
- Quantitative side-by-side cost comparison against Python at fixed θ , to confirm the Julia and Python cost surfaces are numerically equivalent after the bug fixes above.

1.10 Wall-time and configuration

The current end-to-end run on a single RTX 5090 ($T = 14$ days, $h = 15$ min, replan every stride, $n_{\text{SMC}}^{\text{filter}} = 32$ inner-PF chains $\times K = 400$ particles, $n_{\text{SMC}}^{\text{ctrl}} = 1024$ controller chains $\times n_{\text{inner}} = 128$ trials, HMC $\varepsilon = 0.2 / L_{\text{ChEES}} \in \{16, 32, 64, 128, 256\} / \text{num_mcmc}=10$, target nats = 8) completes in 813s. Python’s reference matches this configuration in ~ 27 minutes. The Julia run matches the spec qualitatively (rest-leaning $\Phi \in [0.10, 0.30]$, F clearance from 0.30 to 0.07) but does not yet reproduce the recovery $\rightarrow$ overload pattern that yields the spec’s headline +17% mean- A gain — pending the burst-envelope fix and an independent verification of the cost surface against Python at the gradient level.